

华东师范大学数据学院上机实践报告

课程名称： 操作系统

年级： 2022 级

上机实践成绩：

指导教师： 翁楚良

姓名：

上机实践名称： 实验二彩票调度

学号：

上机实践日期： 2024.4.20

一、实验目的

熟悉调度程序的工作原理，并实现彩票调度算法。

二、实验内容

彩票调度根据进程持有的彩票数量比例，为每个运行的进程分配 CPU。彩票数代表了进程占有某个资源的份额。每个时间片随机抽签决定彩票的赢家。进程拥有的彩票越多，运行的次数就越多。

三、使用环境

VMware Workstation Pro

Visual Studio Code

四、实验过程及结果

• `proc.h` 中修改 `struct proc`，为每个进程添加持有的彩票数 `ticket` 和当前运行的 CPU 时间片 `tick`。

```
37 // Per-process state
38 struct proc {
39     uint sz; // Size of process memory (bytes)
40     pde_t* pgdir; // Page table
41     char *kstack; // Bottom of kernel stack for this process
42     enum procstate state; // Process state
43     int pid; // Process ID
44     struct proc *parent; // Parent process
45     struct trapframe *tf; // Trap frame for current syscall
46     struct context *context; // swtch() here to run process
47     void *chan; // If non-zero, sleeping on chan
48     int killed; // If non-zero, have been killed
49     struct file *ofile[NOFILE]; // Open files
50     struct inode *cwd; // Current directory
51     char name[16]; // Process name (debugging)
52     int ticket; //ticket of process
53     int tick; //tick of process
54 };
```

• `int settickets(int number)` 设置调用进程的彩票数量。默认情况下，每个进程被分配 5 个彩票。获取调用进程的 `pid`，然后遍历 `ptable.proc` 数组，寻找与调用进程 `pid` 相同的进程并将其彩票数 `ticket` 设置为传入的整数参数 `number`，最后释放 `ptable` 的锁。

```
327 int settickets(int number)
328 {
329     struct proc *pr = myproc();
330     int pid = pr->pid;
331     acquire(&ptable.lock); //Find and assign the tickets to the process
332     struct proc *p;
333     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
334     {
335         if(p->pid == pid)
336         {
337             p->ticket = number; //assigning allotted ticket for a process
338             release(&ptable.lock);
339             return 0;
340         }
341     }
342     release(&ptable.lock);
343     return 0;
344 }
```

• `sys_settickets()` 在 `sysproc.c` 中，通过 `argint` 函数从用户空间获取参数。如果彩票数小于 0，则返回 -1 表示不成功；如果彩票数为 0，则返回 `settickets()` 将彩票数设置为初始值；否则将彩票数设置为获取的值。

```
94 int sys_settickets(void)
95 {
96     int number;
97     argint(0, &number);
98     if (number < 0)
99     {
100         return -1; //unsuccessful
101     }
102     else if (number == 0)
103     {
104         return settickets(InitialTickets); //default
105     }
106     else
107     {
108         return settickets(number);
109     }
110 }
```

• `param.h` 中定义初始彩票值 `InitialTickets` 为 5。

```

C param.h
1  #define NPROC      64 // maximum number of processes
2  #define KSTACKSIZE 4096 // size of per-process kernel stack
3  #define NCPU       8 // maximum number of CPUs
4  #define NOFILE     16 // open files per process
5  #define NFILE      100 // open files per system
6  #define NINODE     50 // maximum number of active i-nodes
7  #define NDEV       10 // maximum major device number
8  #define ROOTDEV    1 // device number of file system root disk
9  #define MAXARG      32 // max exec arguments
10 #define MAXOPBLOCKS 10 // max # of blocks any FS op writes
11 #define LOGSIZE     (MAXOPBLOCKS*3) // max data blocks in on-disk log
12 #define NBUF        (MAXOPBLOCKS*3) // size of disk block cache
13 #define FSSIZE      1000 // size of file system in blocks
14 #define InitialTickets 5

```

• `int getpinfo(struct pstat *)` 返回有关所有运行进程的信息，包括 `struct pstat` 的四个成员变量信息。遍历 `ptable.proc` 数组，将每个进程的信息存储到 `pstat` 结构体 `ps` 中，包括进程 `pid`、进程状态、进程彩票数以及时间片，其中 `state` 有 `UNUSED`, `EMBRYO`, `SLEEPING`, `RUNNABLE`, `RUNNING`, `ZOMBIE` 六种状态，统计非 `UNUSED` 状态，最后释放 `ptable` 的锁。

```

346 int getpinfo(struct pstat *ps)
347 {
348     acquire(&ptable.lock);
349     struct proc *p;
350     int i = 0;
351     for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
352     {
353         ps->pid[i] = p->pid;
354         ps->inuse[i] = p->state != UNUSED;
355         ps->ticket[i] = p->ticket;
356         ps->tick[i] = p->tick;
357         i++;
358     }
359     release(&ptable.lock);
360     return 0;
361 }

```

• `pstat.h` 中填充的 `pstat` 结构，存储每个进程的彩票信息的数据结构，并将结果传递到用户空间。包括是否可运行（1 或 0）、进程的彩票数、进程 ID 以及已累积的 CPU 时间片数量。NPROC 表示最大进程数。

```

7  struct pstat
8  {
9      int inuse[NPROC]; // whether this slot of the process table is in use (1 or 0)
10     int ticket[NPROC]; // the number of tickets this process has
11     int pid[NPROC]; // the PID of each process
12     int tick[NPROC]; // the number of ticks each process has accumulated
13 };

```

- `sys_getpinfo()` 中使用 `argptr` 获取用户空间传递的结构体指针。返回 0 表示成功，返回 -1 表示失败，如向内核传递了一个坏指针或 NULL 指针。

```
112 int sys_getpinfo(void)
113 {
114     struct pstat *ps;
115     if(argptr(0, (char **)&ps, sizeof(struct pstat)) < 0)
116         return -1;
117     getpinfo(ps);
118     return 0;
119 }
```

- `int getRunnableProcTickets()` 函数获取所有彩票总数，遍历 `ptable`，将所有 RUNNABLE 状态的进程彩票数 `ticket` 累加得到 `total` 彩票数。

```
363 int getRunnableProcTickets(void)
364 {
365     struct proc *p;
366     int total = 0;
367     for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
368     {
369         if (p->state == RUNNABLE)
370         {
371             total += p->ticket;
372         }
373     }
374     return total;
375 }
```

- `rand.c`、`rand.h` 的实现，将伪随机数生成器添加到了 `xv6` 内核中。

- `proc.c` 中的 `void scheduler()` 实现彩票调度。可运行进程的总彩票数 `total` 由上述函数 `getRunnableProcTickets()` 获得。`rand.c` 中的 `random_at_most()` 函数随机生成一个范围在 0 到总彩票数之间的中奖号码 `win_ticket`。遍历 `ptable`，累加每个进程的彩票数至 `cur_total`，直到累加的彩票数大于中奖号码 (`cur_total > win_ticket`)，然后选定当前进程为累加过程中找到的进程，切换到此进程执行。同时记录该进程执行的时间片并更新进程的时间片信息。

```

C proc.c
377 void
378 scheduler(void)
379 {
380     struct proc *p;
381     struct cpu *c = mycpu();
382     c->proc = 0;
383     for(;;){
384         // Enable interrupts on this processor.
385         sti();
386         long cur_total = 0;
387         // Loop over process table looking for process to run.
388         acquire(&ptable.lock);
389         long total = getRunnableProcTickets() * 1LL;
390         long win_ticket = random_at_most(total);
391         for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
392             //code here!!!!
393             if(p->state == RUNNABLE)
394                 cur_total += p->ticket;
395             else
396                 continue;
397             if(cur_total > win_ticket)
398             {
399                 c->proc = p; // Switch to chosen process. It is the process's job
400                 switchvm(p); // to release ptable.lock and then reacquire it
401                 p->state = RUNNING; // before jumping back to us.
402                 // p->tick++;
403                 int tick_start = ticks;
404                 swtch(&(c->scheduler), p->context);
405                 int tick_end = ticks;
406                 p->tick += (tick_end - tick_start);
407                 switchkvm();
408                 // Process is done running for now.
409                 // It should have changed its p->state before coming back.
410                 c->proc = 0;
411                 break;
412             }
413             else
414                 continue;
415         }
416         release(&ptable.lock);
417     }
}

```

• `allocproc()` 函数中增加 `ticket` 和 `tick` 的初始值。其中新进程彩票数初始化为 `InitialTickets`，时间片初始化为 0。

```

93 //ticket and tick!!!.....
94 p->ticket = InitialTickets;
95 p->tick = 0;

```

• `fork()` 函数中增加代码行，将父进程 `curproc` 的彩票数继承到子进程 `np`，使其共享相同的彩票数。

```

214 np->ticket = curproc->ticket;

```


• `testTicket.c` 中设置当前进程的彩票数。通过接受命令行参数并用 `atoi()` 函数转换为整数，获取要设置的彩票数。使用 `settickets()` 系统调用来设置当前进程的票数为给定的值，主要作用为用户空间调整进程的调度优先级。

```
C testTicket.c
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4  int main(int argc, char *argv[])
5  {
6      int number = atoi(argv[1]); //char->int
7      settickets(number);
8      while(1)
9      {
10         /*just for delay*/
11     }
12     exit();
13 }
```

• `testProcInfo.c` 用于显示当前正在运行的进程信息，包括 `pstat` 中的进程 `pid`、是否可运行、分配的彩票数以及已累积的 CPU 时间片数。通过调用 `getpinfo()` 系统调用来获取进程信息，如果进程存在且彩票数大于 0，则将其打印到标准输出，提供了一个简单的方式来查看系统中各个进程的调度情况和资源利用情况。

```
C testProcInfo.c
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4  #include "pstat.h"
5  #include "param.h"
6  int main(int argc, char *argv[])
7  {
8      int count = 0; //total tick
9      struct pstat ps;
10     getpinfo(&ps);
11     printf(1, "\n----- Initially assigned Tickets = %d ----- \n", InitialTickets);
12     printf(1, "\nProcessID\tRunnable(0/1)\tTickets\t\tTicks\n");
13     for(int i=0; i<NPROC; i++)
14     {
15         if (ps.pid[i] && ps.ticket[i] > 0) //Process exists and ticket>0
16         {
17             printf(1, "%d\t\t%d\t\t%d\t\t%d\n", ps.pid[i], ps.inuse[i], ps.ticket[i], ps.tick[i]);
18             count += ps.tick[i];
19         }
20     }
21     printf(1, "\n----- Total Ticks = %d ----- \n\n", count);
22     exit();
23 }
```

需要修改和补充的文件：

- `defs.h` 中新增 `struct pstat` 来存储运行进程状态的所有信息。

```
C defs.h
1  struct buf;
2  struct context;
3  struct file;
4  struct inode;
5  struct pipe;
6  struct proc;
7  struct rtcdate;
8  struct spinlock;
9  struct sleeplock;
10 struct stat;
11 struct superblock;
12 struct pstat;
```

新增 `settickets()` 和 `getpinfo()` 两个系统调用的声明。

```
105 //PAGEBREAK: 16
106 // proc.c
107 int      cpuid(void);
108 void     exit(void);
109 int      fork(void);
110 int      growproc(int);
111 int      kill(int);
112 struct cpu* mycpu(void);
113 struct proc* myproc();
114 void     pinit(void);
115 void     procdump(void);
116 void     scheduler(void) __attribute__((noreturn));
117 void     sched(void);
118 void     setproc(struct proc*);
119 void     sleep(void*, struct spinlock*);
120 void     userinit(void);
121 int      wait(void);
122 void     wakeup(void*);
123 void     yield(void);
124 int      settickets(int); //Lottery Scheduling
125 int      getpinfo(struct pstat*); //Lottery Scheduling
```

- `proc.c` 中新增两个头文件。

```
C proc.c
1  #include "types.h"
2  #include "defs.h"
3  #include "param.h"
4  #include "memlayout.h"
5  #include "mmu.h"
6  #include "x86.h"
7  #include "proc.h"
8  #include "spinlock.h"
9  #include "pstat.h"
10 #include "rand.h"
```

- `sysproc.c` 中新增头文件。

```
C sysproc.c
1  #include "types.h"
2  #include "x86.h"
3  #include "defs.h"
4  #include "date.h"
5  #include "param.h"
6  #include "memlayout.h"
7  #include "mmu.h"
8  #include "proc.h"
9  #include "pstat.h"
```

- `syscall.c` 中新增 `sys_settickets` 设置调用进程的彩票数和 `sys_getpinfo` 获取所有运行进程的信息。

```
C syscall.c
106 extern int sys_settickets(void); //
107 extern int sys_getpinfo(void); //
```

```
131 [SYS_settickets] sys_settickets,
132 [SYS_getpinfo] sys_getpinfo,
```


- syscall.h 中定义系统调用的编号。

```
C syscall.h
1 // System call numbers
2 #define SYS_fork 1
3 #define SYS_exit 2
4 #define SYS_wait 3
5 #define SYS_pipe 4
6 #define SYS_read 5
7 #define SYS_kill 6
8 #define SYS_exec 7
9 #define SYS_fstat 8
10 #define SYS_chdir 9
11 #define SYS_dup 10
12 #define SYS_getpid 11
13 #define SYS_sbrk 12
14 #define SYS_sleep 13
15 #define SYS_uptime 14
16 #define SYS_open 15
17 #define SYS_write 16
18 #define SYS_mknod 17
19 #define SYS_unlink 18
20 #define SYS_link 19
21 #define SYS_mkdir 20
22 #define SYS_close 21
23 #define SYS_settickets 22
24 #define SYS_getpinfo 23
```

- usys.S 中新增两个 SYSCALL。

```
ASM usys.S
29 SYSCALL(syscall)
30 SYSCALL(sleep)
31 SYSCALL(uptime)
32 SYSCALL(settickets)
33 SYSCALL(getpinfo)
```

- user.h 中新增系统调用的声明。

```
C user.h
25 int sleep(int);
26 int uptime(void);
27 int settickets(int);
28 int getpinfo(struct pstat*);
```

- Makefile 中在 OBJS 变量中新增 rand.o 文件，在 UPROGS 变量中新增 _testTicket 和 _testProcInfo 两个用户程序。

```
M Makefile
1  OBJS = \
20  trap.o\
27  uart.o\
28  vectors.o\
29  vm.o\
30  rand.o\
```

```
M Makefile
169  UPROGS=\
182  _usertests\
183  _wc\
184  _zombie\
185  _testTicket\
186  _testProcInfo\
```

运行结果：

```
init: starting sh
$ testTicket 10&; testTicket 15&; testTicket 20&; testTicket 25&; testTicket 30&;
$ testProcInfo

----- Initially assigned Tickets = 5 -----
ProcessID      Runnable(0/1)  Tickets  Ticks
1              1              5        5
2              1              5        3
14             1              5        0
5              1              10       105
7              1              15       156
9              1              20       193
11             1              25       239
13             1              30       280

----- Total Ticks = 981 -----
$ █
```

可以观察到票数多的进程执行时间的确会更长。

五、总结

通过本次实验，我熟悉了操作系统的调度程序工作原理，并实现了彩票调度算法。每个时间片随机抽出彩票赢家并运行，进程拥有彩票越多，运行次数就越多。运用了彩票调度的基本思想，通过生成随机值，做到按比例分配 CPU 时间片，并实现了设置彩票数量和返回进程信息的两个系统调用，进一步理解并掌握了调度算法的工作机制。